Financial Toolkit

Hugo Ballestero Javier Arnanz Diego Antón

Máster MIAX · BME Institute · Taller 1

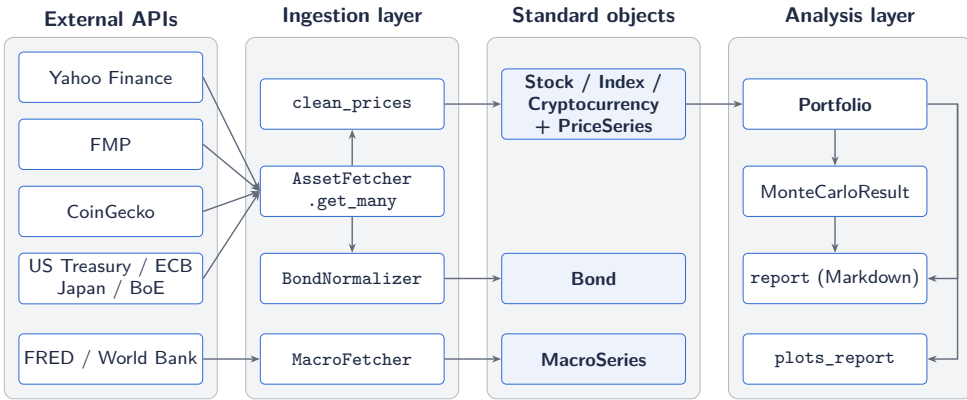
10/11/2026

Qué es

- Librería de Python instalable: `financial_toolkit`
- **Ingesta** desacoplada del **análisis**: todas las fuentes devuelven las mismas *dataclasses*
- Añadir una API o un análisis no obliga a tocar el núcleo

Área	**Fuentes / contenido**
Acciones e índices	Yahoo Finance, FMP
Criptomonedas	Yahoo Finance, CoinGecko
Fundamentales	Los 3 estados financieros
Tipos de interés	EE. UU., BCE, Japón, BoE
Macro	FRED, World Bank
Análisis	Cartera, Monte Carlo, informe

Data flow



El análisis solo conoce los objetos estándar, **nunca la respuesta cruda de una API**.

Código: N activos de cualquier fuente en una llamada

```
requests = [
    {"type": "stock", "api": "yfinance", "symbol": "AAPL"},
    {"type": "stock", "api": "fmp", "symbol": "MSFT"},
    {"type": "crypto", "api": "coingecko", "symbol": "bitcoin"},
    {"type": "index", "api": "yfinance", "symbol": "^GSPC"},
    {"type": "bond", "api": "bce", "symbol": "EU10Y"},
]
fetcher = AssetFetcher()
assets = fetcher.get_many(requests, start="2023-01-03", end="2025-10-31")
fetcher.cleaning_reports["AAPL"]   # CleaningReport for each asset
```

Yahoo y FMP devuelven la **misma clase** `Stock` (diferencia media de cierre: 0,0009 %), y CoinGecko devuelve `Cryptocurrency` con sus `Tokenomics`.

Código: qué hace `get_many()` por dentro (simplificado)

```
fetchers = {                                    # 10 (type, api) combinations
    ("stock", "yfinance"): self.yahoo.get_stock,
    ("crypto", "coingecko"): self.coingecko.get_crypto,
    ("bond", "bce"): self.bce.get_data,
}
for request in requests:
    asset = fetchers[(kind, api)](**parameters)
    if kind == "bond":
        asset = self.bond_normalizer.normalize(data=asset, ...)
    else:
        prices, report = clean_prices(self._trim(asset, start, end).price)
```

- **Añadir una API = añadir una entrada** al diccionario: el resto del código no cambia
- Las acciones, índices y criptos se recortan al periodo y se limpian; los bonos se normalizan a Bond

Código: PriceSeries, fundamentales y macro

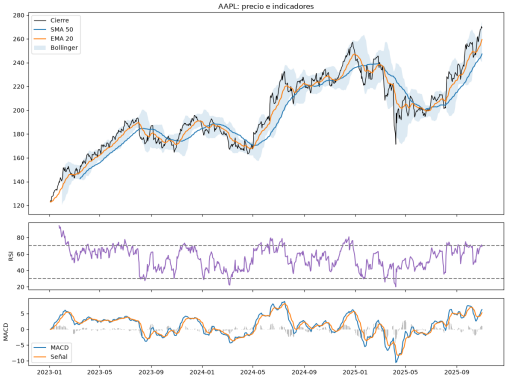
```
prices = aapl.price                                  # PriceSeries
prices.annual_return, prices.annual_volatility    # 27.8 %, 25.9 %: computed on creation
prices.sma(50), prices.ema(20), prices.bollinger_bands()
prices.rsi(), prices.macd(), prices.max_drawdown()          # drawdown -33.4 %

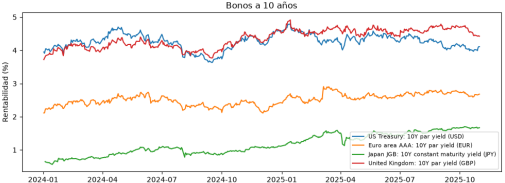
aapl.company.fundamentals.analyze_income_statement()  # margins, growth, warnings

macro = MacroFetcher()
bill = macro.get_macro_series("DTB3", source="FRED", start=START, end=END)
risk_free = float(bill.data.dropna().iloc[-1]) / 100     # 3.73 %
```

Los estadísticos se calculan **al crear el objeto**; los indicadores son métodos. El tipo libre de riesgo de FRED alimenta después el Sharpe.

Gráficos: indicadores de AAPL y bonos a 10 años





Código: datos honestos y validación estricta

```
clean, report = clean_prices(dirty_series)        # 2 missing closes + an unadjusted split
# CleaningReport(n_input=250, n_output=248, ..., missing=2, outliers=1)
clean_prices(too_dirty_series)                     # 10 % of the closes missing
# ValueError: missing closes must be at most 5%, got 10.00% (25 of 250)

Portfolio.from_weights([san_eur], [1.0], 10_000, date)
# ValueError: Convert asset prices to USD before trading
Portfolio.from_weights([bond], [1.0], 10_000, START)
# TypeError: Invalid asset type
```

- Los cierres inválidos **se eliminan, no se rellenan**; los outliers **se cuentan, no se tocan**
- Mejor un **error claro** que una métrica falsa

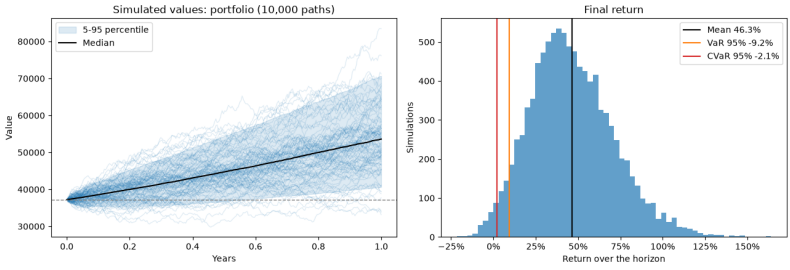
Código: cartera y Monte Carlo

```
weights = {"AAPL": 0.25, "MSFT": 0.25, "SAN": 0.15, "IEF": 0.20, "BTC-USD": 0.15}
portfolio = Portfolio.from_weights([aapl, msft, san, ief, btc], weights, 10_000, START)
portfolio.buy(aapl, 10, "2024-01-03")            # paid with cash
portfolio.deposit(5_000, "2024-06-03")           # external flow, not a gain
portfolio.rebalance([aapl, msft, san, ief, btc], weights, "2025-01-02")

simulation = portfolio.monte_carlo(10_000, years=1, seed=42)
stress = portfolio.monte_carlo(10_000, years=1, annual_return=0.03, annual_volatility=0.35)
portfolio.report(risk_free=risk_free, monte_carlo=simulation)    # Markdown
```

Aportado	Valor final	TWR	Volatilidad	Sharpe	Máx. drawdown
15.000 USD	**37.198 USD**	195,7 %	16,8 %	**2,04**	−14,2 %

Gráfico: Monte Carlo a 1 año



	Histórico	Estrés 3 %/35 %	Solo BTC
Rentabilidad media	46,3 %	9,9 %	119,4 %
Mediana	44,1 %	3,0 %	94,5 %
Prob. de pérdida	1,6 %	**46,7 %**	8,4 %

Gráfico: `plots_report()`



Conclusiones

- **Diseño en capas**: añadir una API o un análisis no obliga a tocar el resto
- **Salida estandarizada**: Yahoo, FMP, CoinGecko y 4 bancos centrales producen los mismos objetos
- **Datos honestos**: no se rellenan precios, los outliers se avisan y las métricas usan solo fechas observadas
- **Validación estricta**: errores claros antes que números falsos

¡Gracias!